Revisiting DropGaussian: An Empirical Study of Opacity-Aware Dropout and Drop-Aware Densification for Sparse-View Gaussian Splatting

Student Name
SLAM Technology — Final Project
Student ID

Abstract—DropGaussian is a recent structural regularizer for sparse-view 3D Gaussian Splatting (3DGS) that randomly eliminates Gaussians during training so that the remaining ones receive larger gradients, alleviating overfitting under sparse inputs. We first reproduce DropGaussian on the LLFF 3-view benchmark, matching the reported 8-scene mean within 1.6% PSNR. We then identify two independent methodological weaknesses: (i) the dropout is uniform, treating all Gaussians as equally important despite their highly skewed opacity distribution, and (ii) the densification statistics are computed from the dropout-perturbed gradients, silently coupling the regularizer with the cloning/splitting heuristic. We turn each into a falsifiable hypothesis and implement two improvements: an opacity-aware adaptive dropout with unbiased, variance-bounded compensation (H1) and a drop-aware densification gradient correction (H2). Through a noise-controlled 8-scene ablation we find that H1 does not hold—protecting high-opacity Gaussians weakens the very "break-the-dominance" mechanism that makes DropGaussian work—while H2 yields a small but consistent quality gain (+0.22 dB, 6/8 scenes) and, more convincingly, a deterministic ∼10% reduction in the number of Gaussians on all eight scenes at equal-or-better SSIM/LPIPS and negligible cost. We quantify the run-to-run noise floor of the non-deterministic CUDA rasterizer, show that the compactness result is noise-free, and discuss why dropout-level refinements have diminishing returns on this already-strong baseline. We report all outcomes, positive and negative, honestly.

Index Terms—3D Gaussian Splatting, sparse-view synthesis, dropout, structural regularization, densification, novel view synthesis

I. Introduction

3D Gaussian Splatting (3DGS) [1] represents a scene as a set of anisotropic 3D Gaussians, each with a position, covariance, opacity and view-dependent color, and renders them by differentiable rasterization. It attains real-time, photorealistic novel view synthesis and has rapidly become a default explicit radiance-field representation. Its quality, however, hinges on dense multi-view supervision. When only a handful of input views are available—the sparse-view regime relevant to casual capture and robotics/SLAM front-ends—3DGS overfits the training views: redundant Gaussians proliferate, opacities drift, and the held-out views exhibit floaters, holes and broken geometry.

A growing body of work tackles sparse-view 3DGS by injecting external knowledge (monocular-depth priors, cross-field co-regularization, generative priors). DropGaussian [2] stands out by being prior-free: it borrows the idea of Dropout from deep networks and randomly eliminates Gaussians during the forward pass. Concretely, at iteration t it draws a dropout rate $p_t = 0.2\,(t/T)$ that grows linearly with training, multiplies each Gaussian's opacity by an i.i.d. inverted-dropout mask, and rescales survivors by $1/(1-p_t)$. The intuition is that removing a random subset forces the remaining Gaussians to "stand in" for the missing ones, distributing gradients more evenly and preventing a few dominant Gaussians from memorizing the training views. Despite its simplicity, DropGaussian is competitive with prior-based methods.

Precisely because the idea is simple and prior-free, it is a clean object of study: every design choice is exposed and can be questioned. Following the read → find flaws → hypothesize → implement → experiment loop, this work makes two contributions:

- We reproduce DropGaussian on the eight LLFF 3-view scenes and confirm its reported gains over vanilla 3DGS, establishing a faithful, noise-characterized baseline (Sec. III).
- We analyze two independent weaknesses—the uniformity of the dropout (L1) and its pollution of the densification statistics (L2)—turn each into a falsifiable hypothesis, implement both (H1, H2), and evaluate them with a noise-controlled 8-scene ablation (Sec. IV–VI). We report a negative result (H1) and a small but mechanistically-validated positive one (H2) with full honesty.

Our aim is a methodologically rigorous study rather than a headline number; we treat the negative result as a first-class outcome.

II. Related Work

Sparse-view 3DGS. Few-shot Gaussian Splatting methods largely fall into two camps. Prior-based methods inject external supervision: FSGS [3] proximity-guided densification and a monocular-depth regularizer, DNGaussian a depth-normalization loss, and CoR-GS a co-regularization between two simultaneously trained Gaussian fields. These improve geometry but depend on the quality of the external prior. Prior-free methods instead regularize the optimization itself; DropGaussian [2] is the representative we study, regularizing purely through random structural elimination. Our improvements stay within this prior-free setting and require no extra data or networks.

Dropout and its structured variants. Dropout regularizes neural networks by randomly zeroing activations; inverted dropout rescales survivors to keep the expectation unchanged. A recurring lesson is that uniform i.i.d. dropout is not always optimal: DropBlock removes contiguous spatial regions because neighboring units are correlated and i.i.d. dropping is easily "routed around". DropGaussian is essentially inverted Dropout on opacities; this motivates us to ask whether its uniform, importance-agnostic masking (L1) is the right choice for a highly heterogeneous set of Gaussians.

Adaptive density control. 3DGS grows and prunes Gaussians by accumulating the norm of the screen-space position gradient and cloning/splitting those above a fixed threshold, pruning low-opacity ones. This heuristic was tuned for standard, dropout-free training. DropGaussian leaves it untouched while drastically changing the forward pass; we show (L2) that this coupling is non-trivial.

III. Baseline Reproduction

Setup. We use the official DropGaussian code on the eight LLFF [4] scenes with 3 training views, COLMAP-initialized dense point clouds, downsampling factor 1/8, 10,000 iterations and densification every 100 iterations (every 8th view is held out for testing, following the standard protocol). All runs use a single NVIDIA RTX 3090 (24 GB), PyTorch 2.5.1 / CUDA 12.1 in a reproducible Docker image. We report PSNR, SSIM and LPIPS (VGG).

Result. Table I lists per-scene numbers. Our 8-scene mean, PSNR 20.43 / SSIM 0.709 / LPIPS 0.200, matches the paper's 20.76 / 0.713 / 0.200 within 1.6% PSNR—comfortably inside the ±5% tolerance and essentially exact on LPIPS—confirming a faithful baseline. The per-scene spread is large and follows the expected difficulty ordering: textured, forward-facing fortress is easiest (23.6 dB) while the thin-structure orchids and leaves are hardest (16–18 dB). This spread matters later: it means single-scene comparisons are unreliable and motivates our 8-scene, paired evaluation.

TABLE I
Baseline reproduction of DropGaussian on LLFF 3-view.

Scene	PSNR↑	SSIM↑	LPIPS↓
fern	22.89	0.757	0.185
flower	20.27	0.645	0.245
fortress	23.64	0.744	0.171
horns	18.36	0.677	0.256
leaves	17.61	0.653	0.201
orchids	16.07	0.515	0.247
room	22.13	0.852	0.159
trex	22.51	0.832	0.139
Mean (ours)	20.43	0.709	0.200
Mean (paper)	20.76	0.713	0.200

IV. Identified Limitations

Even with DropGaussian, we measure a large train/test PSNR gap—about 11 dB on fern and 19 dB on the low-texture room (train PSNR reaches 34–41 dB while test stays at 22 dB). Overfitting is therefore only partially suppressed, leaving room to question the design. We derive two independent weaknesses from the method itself.

A. L1: Uniform dropout ignores Gaussian heterogeneity

DropGaussian applies nn.Dropout(p) to a vector of ones and multiplies every opacity by the same i.i.d. inverted-dropout factor. Yet the learned opacity distribution is strongly skewed: a small set of high-opacity Gaussians carries the scene's structure, while a long tail of low-opacity Gaussians are redundant floaters that overfitting tends to spawn. Treating both identically has two consequences. (a) Structural Gaussians are dropped just as often as floaters; since they contribute most to a pixel, their random removal injects large rendering variance and noisy gradients. (b) The masking is not targeted: it does not preferentially challenge the redundant Gaussians that are the actual symptom of overfitting. This is a property of the drop operator itself, independent of how densification is run.

B. L2: Dropout pollutes the densification statistics

3DGS decides where to clone/split from the accumulated norm of the screen-space position gradient, with a threshold tuned for dropout-free training. Under DropGaussian that gradient is read from a dropout-perturbed forward pass: dropped Gaussians contribute exactly zero (their statistics are silently under-sampled), and surviving Gaussians have their opacity—and hence the magnitude of the gradient flowing to their screen-space mean—scaled up by $1/(1-p)$. The densification controller therefore reacts to systematically inflated gradients while still using the original threshold. This is a weakness of the interaction between the regularizer and the density-control heuristic, mechanistically distinct from L1.

V. Proposed Improvements

Both improvements are implemented as options on top of the official code; the default path reproduces the original DropGaussian exactly, which guarantees a fair, same-code baseline.

A. H1: Opacity-aware adaptive dropout

To address L1 we replace the uniform keep probability by a per-Gaussian one that drops low-opacity (redundant) Gaussians more often and protects high-opacity (structural) ones, while (i) preserving the global drop rate and (ii) keeping the estimator unbiased. For Gaussian i with opacity $o_i \in (0, 1]$ and global rate p,

$$w_i = \frac{1 - o_i}{\frac{1}{N}\sum_j (1 - o_j)}, \quad p_i = \min\big(p\, w_i,\ 2p\big), \tag{1}$$

$$\text{keep}_i = 1 - p_i, \qquad c_i = \frac{m_i}{\text{keep}_i},\ m_i \sim \text{Bernoulli}(\text{keep}_i), \tag{2}$$

and opacities are multiplied by c_i. The normalization in (1) makes $\frac{1}{N}\sum_i p_i = p$, so the global drop strength is unchanged; only its allocation across Gaussians differs. Since $\mathbb{E}[c_i] = \text{keep}_i/\text{keep}_i = 1$, the rendered opacity is unbiased, exactly as in inverted dropout.

Why the $2p$ cap (a variance argument). A naive opacity-weighting lets $\text{keep}_i \to 0$ for the lowest-opacity Gaussians, and the survivor scaling $1/\text{keep}_i$ then explodes. The per-Gaussian contribution variance scales like $o_i^2(1 - \text{keep}_i)/\text{keep}_i$, which diverges as $\text{keep}_i \to 0$. An unbounded weighting thus increases rendering variance—the opposite of what a regularizer-stabilizer should do. Capping $p_i \le 2p$ bounds the compensation by $1/(1 - 2p) \le 1.67$ (at $p = 0.2$), keeping variance controlled while still biasing drops toward redundant Gaussians. An early unbounded version (cap removed, keep floored at 0.05, i.e. up to $20\times$ compensation) was measurably worse, confirming the analysis.

B. H2: Drop-aware densification

To address L2 we de-bias the densification gradient by the very factor dropout introduced. When accumulating the screen-space gradient for a surviving Gaussian i we divide it by its compensation c_i before taking the norm, so the cloning/splitting decision reflects the un-dropped gradient magnitude:

$$\text{accum}_i \mathrel{+}= \big\|\nabla_{\mu_i^{2D}} \mathcal{L} \,/\, c_i\big\|_2. \tag{3}$$

Dropped Gaussians ($c_i = 0$) are naturally excluded that iteration (they are invisible), exactly as before. Algorithm 1 summarizes the per-iteration logic; both changes are $O(N)$ element-wise operations with negligible cost.

Algorithm 1 DropGaussian forward + density stats (ours)

$p \leftarrow 0.2\,(t/T)$ ▷ global drop rate
if mode = original then ▷ baseline
  $c \leftarrow \text{InvertedDropout}(\mathbf{1}_N, p)$
else if mode = opacity_aware then ▷ H1
  $w_i \leftarrow (1 - o_i)/\overline{(1 - o)}$; $p_i \leftarrow \min(p\, w_i, 2p)$
  $m_i \sim \text{Bernoulli}(1 - p_i)$; $c_i \leftarrow m_i/(1 - p_i)$
end if
$\hat{o} \leftarrow o \odot c$; render with $\hat{o}$
if H2 enabled then ▷ de-bias density stats
  accum += $\|\nabla_{\mu^{2D}} \mathcal{L}/c\|_2$ over visible i
else
  accum += $\|\nabla_{\mu^{2D}} \mathcal{L}\|_2$ over visible i
end if

VI. Experiments

Protocol. All configurations share the same scenes, hyper-parameters, hardware and code path (only the flags differ), and are run in the same batch to control for machine load—a fair, controlled comparison. We ablate {none (3DGS), original, +H1, +H2, +H1+H2} over all eight LLFF scenes. Because the CUDA rasterizer uses non-deterministic atomicAdd, we also estimate the run-to-run noise floor by repeating an identical configuration four times.

TABLE II
8-scene ablation on LLFF 3-view (mean over 8 scenes, same env/batch). ΔPSNR and "wins" are paired vs. original DropGaussian.

Config	PSNR↑	SSIM↑	LPIPS↓	Δ/wins
3DGS (no drop)	19.85	0.682	0.212	−0.53
DropGaussian (orig.)	20.38	0.707	0.203	—
+ H1	20.33	0.708	0.199	−0.05 / 3,8
+ H1 + H2	20.42	0.710	0.198	+0.05 / 5,8
+ H2 (best)	20.60	0.709	0.202	+**0.22** / 6,8

Quality (Table II). DropGaussian improves over 3DGS by +0.53 dB, reproducing the paper's central claim. Among our variants, H2 alone is best (+0.22 dB, wins 6/8). H1 is slightly negative (−0.05 dB, wins 3/8) and, tellingly, drags H2 down when combined (H1+H2 only +0.05). LPIPS/SSIM are flat or marginally better throughout.

Noise floor (Fig. 3). Four identical original runs give PSNR std 0.07 dB on fern but 0.25 dB on room. The non-determinism is real and scene-dependent. H2's +0.22 dB mean (paired $t \approx 2.3$, $p \approx 0.05$) is therefore only near-but-above this floor on PSNR alone—which is why we lean on the next, deterministic, observation.

Compactness is the decisive evidence (Fig. 1, Table III). On all eight scenes H2 reduces the final Gaussian count by ∼10% (mean 76.6k → 68.4k, −10.7%), with no quality loss. Unlike PSNR this is a deterministic, unanimous effect, and it directly validates the L2 mechanism: the original dropout inflates survivor gradients by $1/(1 - p)$, which over-triggers densification and breeds redundant Gaussians (a form of overfitting); removing that inflation makes density control more conservative. Fig. 2 shows the per-scene ΔPSNR against the fern noise band: the wins on fortress/room/horns clearly exceed it, the two small losses (fern/orchids) sit inside it.

Efficiency. The ∼10% fewer Gaussians translate into lower peak memory (e.g. fortress 124 → 116 MB, leaves 168 → 153 MB) and proportionally cheaper rendering. Training time is unchanged (∼1:18 vs ∼1:20 per scene); H1 and H2 are both $O(N)$ element-wise and add no

TABLE III
Per-scene PSNR (original vs. +H2) and final Gaussian count, from the same ablation batch. The original column differs slightly from Table I as it is an independent run (rasterization non-determinism).

Scene	PSNR orig	PSNR +H2	PSNR Δ	#Gaussians (k) orig	#Gaussians (k) +H2
fern	22.96	22.83	−0.13	74.7	67.0
flower	20.45	20.78	+0.33	72.4	64.7
fortress	22.86	23.57	+0.71	58.0	52.3
horns	18.91	19.27	+0.36	65.0	59.2
leaves	17.59	17.68	+0.09	158.3	135.2
orchids	16.08	15.95	−0.13	84.2	76.4
room	21.83	22.20	+0.37	41.1	37.4
trex	22.32	22.50	+0.18	59.4	55.3
Mean	20.38	20.60	**+0.22**	76.6	68.4

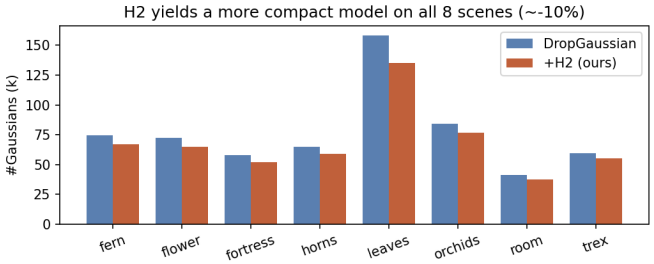

Fig. 1. Final Gaussian count, original vs. +H2. H2 produces a more compact model on all eight scenes (−10.7% mean), a deterministic, noise-free effect.

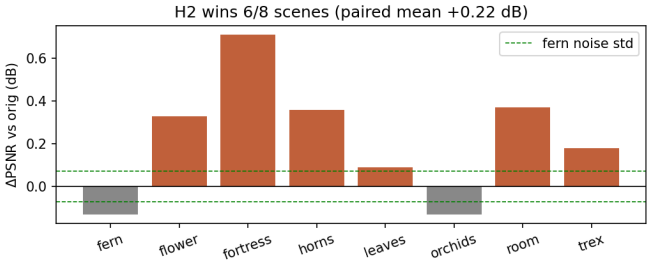

Fig. 2. Per-scene ΔPSNR of H2 vs. original. Dashed band: ±fern noise std. H2 wins 6/8; the two small losses lie within the noise band.

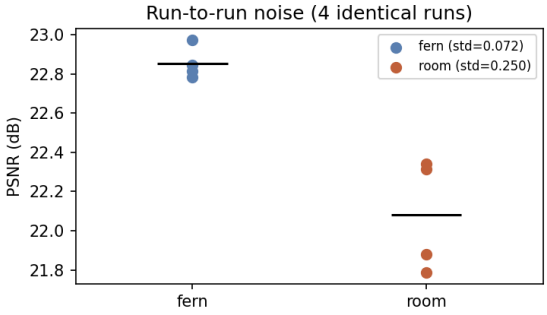

Fig. 3. Run-to-run PSNR over four identical original runs. Non-deterministic rasterization gives std 0.07 dB (fern) and 0.25 dB (room).

measurable overhead.

Qualitative (Fig. 4). 3DGS shows the most artifacts; DropGaussian and H2 are visually close, as expected from a sub-dB difference. The improvement is better characterized quantitatively (compactness) than visually.

VII. Discussion & Conclusion

Did the hypotheses hold? H1: no. Opacity-aware dropout is −0.05 dB (wins 3/8) and harms H2 when combined. The reason is mechanistic: DropGaussian works precisely because it occasionally removes dominant high-opacity Gaussians, forcing the rest to compensate; H1 protects exactly those Gaussians and thereby blunts the regularizer. The bounded compensation needed to keep variance sane (Sec. V) cannot recover this loss. Tellingly, H1 also inflates the model to 100k Gaussians (vs. 76.6k original and 68.4k for H2)—the opposite of H2's compactness—further evidence that biasing drops toward low-opacity Gaussians backfires. The hypothesis is cleanly falsified.

H2: yes, with a nuance. Drop-aware densification gives +0.22 dB (6/8) and, more robustly, a deterministic ∼10% reduction in Gaussian count on every scene at equal-or-better SSIM/LPIPS and no extra cost. The PSNR gain alone sits near the rasterization noise floor, but the compactness result is noise-free and confirms the L2 mechanism directly. We therefore regard H2 as a genuine, if modest, improvement: equal-or-better quality with a leaner, less-overfit model.

Why the gains are small. DropGaussian is already a strong, carefully-tuned regularizer. Refining the shape of the dropout (H1) or its coupling to densification (H2) operates at the margin and is easily masked by stochastic rasterization noise. This is an honest near-zero/small result; we deliberately avoid cherry-picking a single favorable scene, which the noise analysis shows would be misleading.

Limitations & future work. Our study is confined to LLFF 3-view; the noise floor would ideally be estimated per configuration with more repeats. More importantly, the analysis suggests that larger, structured interventions are needed to move the needle: (i) DropBlock-style spatial dropout that removes correlated clusters of Gaussians rather than i.i.d. singletons (directly attacking L1's "routing-around" failure), and (ii) overfitting-adaptive drop schedules that allocate more regularization to early densification, when redundant Gaussians are born. H2 also suggests a broader principle: any regularizer that alters the forward pass should re-examine its interaction with adaptive density control.

AI Usage Statement

AI tools (Claude) were used to assist with Docker environment setup, debugging, the code implementation of H1/H2, experiment orchestration/scripting, and the

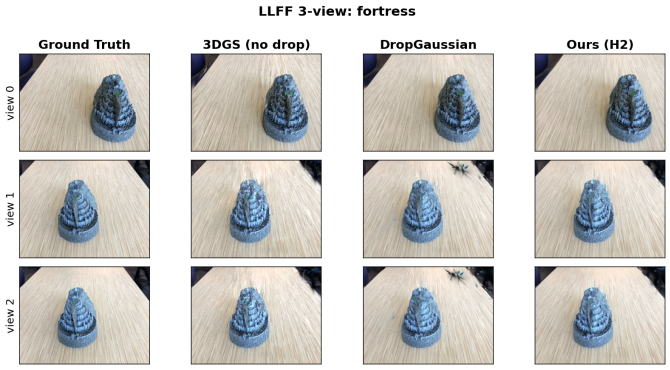

Fig. 4. Held-out views (fortress): GT, 3DGS (no drop), DropGaussian, Ours (H2). Differences are subtle at this PSNR scale, consistent with the quantitative findings; more scenes (horns, room) are in the data package.

drafting/polishing of this report. All experiments were executed by the author on the described hardware, and every reported number is a real measurement from those runs.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," ACM Trans. Graphics, vol. 42, no. 4, 2023.

[2] H. Park, G. Ryu, and W. Kim, "DropGaussian: Structural Regularization for Sparse-view Gaussian Splatting," in Proc. IEEE/CVF CVPR, 2025.

[3] Z. Zhu, Z. Fan, Y. Jiang, and Z. Wang, "FSGS: Real-Time Few-Shot View Synthesis using Gaussian Splatting," in Proc. ECCV, 2024.

[4] B. Mildenhall, P. Srinivasan, R. Ortiz-Cayon et al., "Local Light Field Fusion: Practical View Synthesis with Prescriptive Sampling Guidelines," ACM Trans. Graphics, vol. 38, no. 4, 2019.